

LIVE EVENTS · TICKETING

GATE

The poster is the ground, and the palette is read out of it in code — every event re-themes the whole app. Nothing here was picked from a swatch.

SCOPE

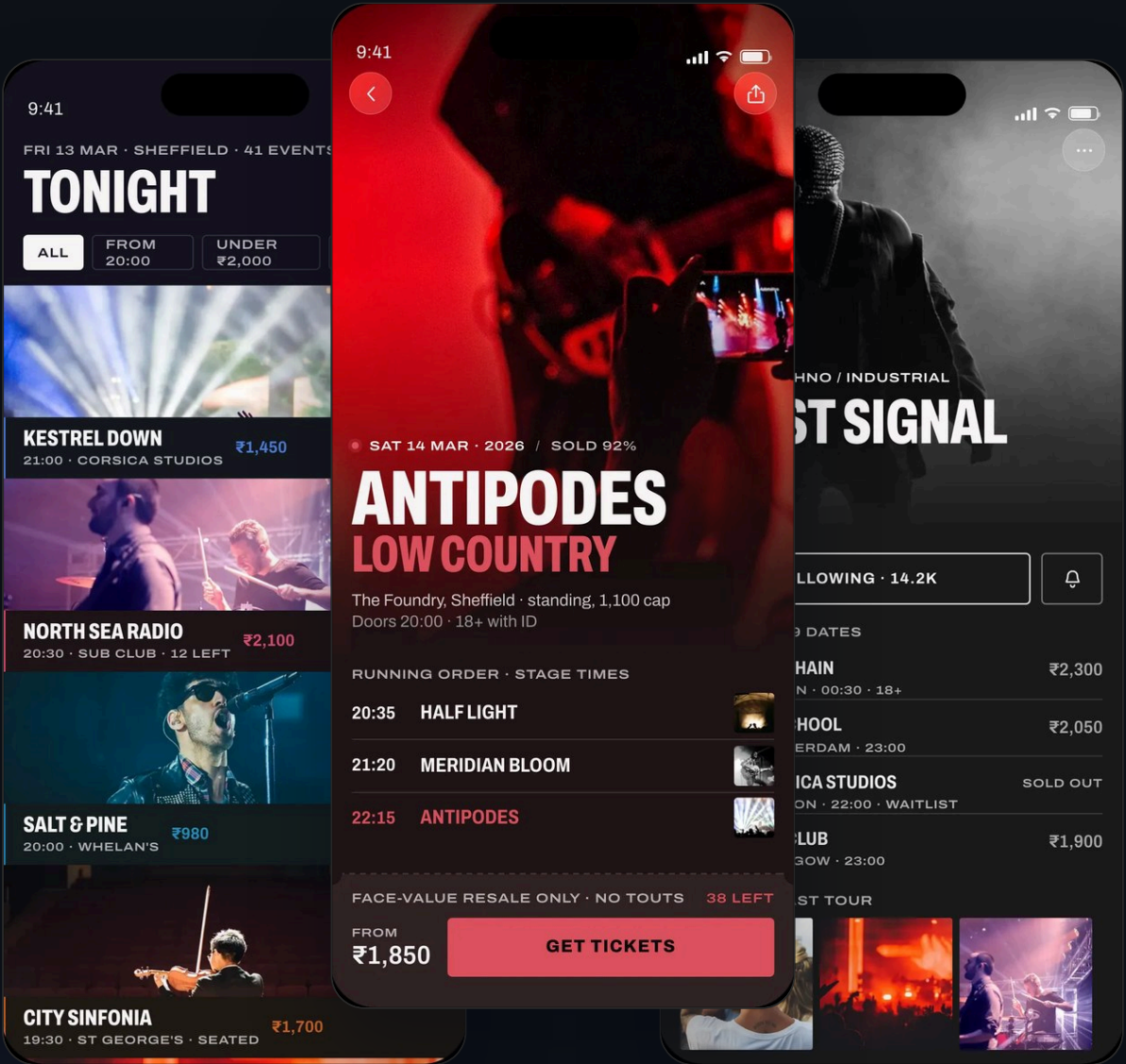
7 screens, built and shot on device

STACK

SwiftUI · Swift 6 · async/await · iOS 26

STANDARD

Light + dark · Dynamic Type · VoiceOver



- Swift
- SwiftUI
- async/await
- Core Image
- PassKit

Self-initiated demo build. Designed and coded end to end by me to show current iOS 26 / SwiftUI standards — not client work, and no client material appears in it.

11 yrs

iOS engineering

Fintech

Banking-grade discipline

SwiftUI

iOS 26 · Swift 6

7

Screens in this build

● The problem

Ticketing apps put the artwork in a thumbnail and paint everything around it in the company's own brand colour, so every gig — a string quartet and a warehouse techno night — arrives looking identical. Then the booking fee shows up on the last screen.

● What I built

The poster runs edge to edge as the ground, and the palette is extracted from it rather than designed: quantize the plate, discard the near-greys, weight what is left by pixel count $\times$ saturation $^{1.5}$. Change the poster and the app re-themes. The fee is on the checkout screen, above the total.

02 Engineering decisions

- ✓ Per-event palette derived in code — ten events, ten grounds and accents, none chosen by hand
- ✓ Accent lightness solved for a fixed 4.6:1 against that event's own ground, so amber and blue carry identical weight; source chroma is a stage light, not a UI token
- ✓ Monochrome posters take an explicit zero-chroma branch to silver instead of falling through to red
- ✓ One family, Archivo, split by its width axis — heads at width 62, labels at width 122
- ✓ No tab bar on any screen; one die-cut ticket with punched notches is the app's only container
- ✓ Face-value return queue drawn as a real state, with a live waitlist position

Swift

SwiftUI

async/await

Core Image

PassKit

Vision

Extracted palette, no fixed accent

● What this transfers to your build

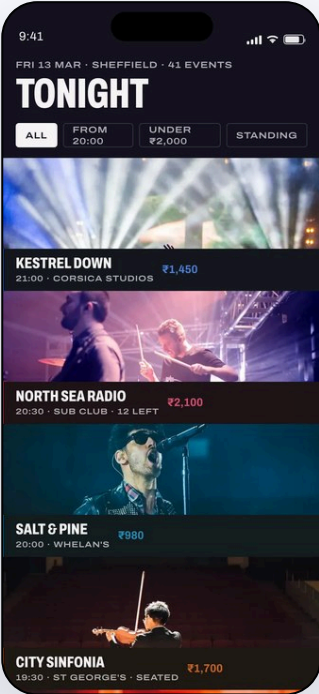
If your app is themed by content — events, brands, teams, listings — this is the pattern: colour derived in code from the artwork, contrast solved rather than eyeballed, and a checkout that shows fees before the total. The palette extractor and the PassKit wallet flow drop into any ticketing, booking or membership app.

03 Every screen, shipped

7 screens, built in SwiftUI and captured on device. Light and dark, Dynamic Type and VoiceOver labels are wired on each one.



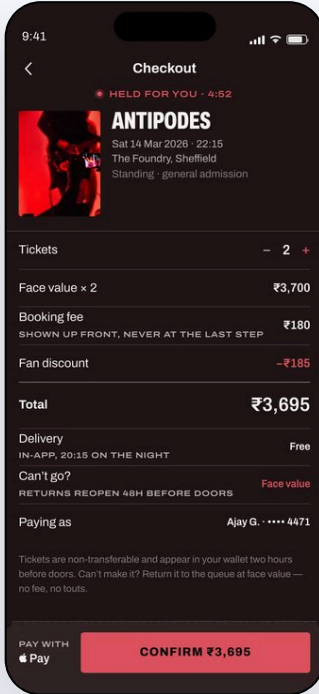
Event with running order



Tonight — five events, five palettes



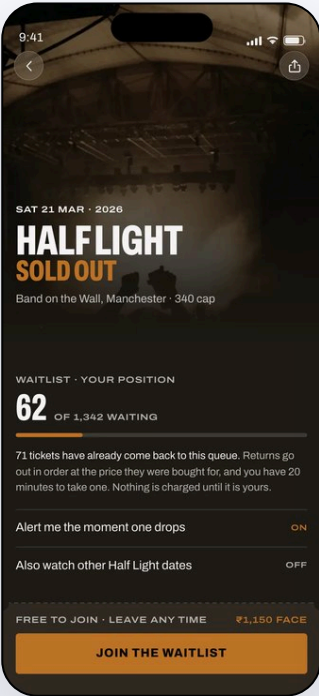
Artist page



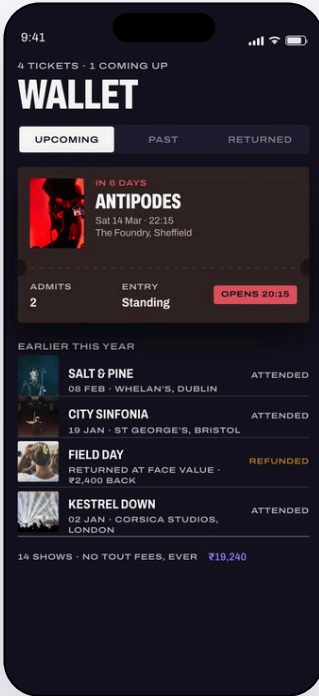
Checkout with a held-inventory timer



Live ticket, die-cut stub



Sold out — face-value waitlist



Wallet

04 What a build like this gets you

- ✓ Production SwiftUI screens, not clickable prototypes — code your team can ship
- ✓ MVVM + Clean Architecture with a design token layer: colors, type, spacing, radii
- ✓ async/await networking, actors and structured concurrency. No third-party UI deps
- ✓ Light and dark, Dynamic Type, VoiceOver labels and reduced-motion paths from day one

05 How we can work together

Screen build

3 screens

6 days

3 SwiftUI screens from your designs, with the token layer and mock data.

MOST PICKED

Feature or MVP

6+ screens

12 days

6+ screens, MVVM, navigation stack, async/await API layer, light and dark.

Full app or rescue

End to end

3 weeks+

End to end build, or an audit and rescue of an inherited codebase, to App Store.

06 Also on the table

- UIKit / Objective-C → SwiftUI migration, Swift 6 strict concurrency
- On-device AI with Apple Foundation Models, Core ML, Vision
- Claude and OpenAI integrations with streaming, cost control and privacy
- StoreKit 2 subscriptions, Widgets, Live Activities, App Intents and Siri
- Crash, performance and App Store rejection fixes on an existing app
- XCTest, Fastlane, Xcode Cloud and GitHub Actions CI/CD

07 How the build runs

DAY 0

Requirements

You send designs or a reference app, target iOS version and API docs. The clock starts here, not the day we agree to work together.

DAY 2

Foundation

Design tokens, navigation shell and data models land. You get a build you can run on your own device.

DAY 6

Screens

Feature screens built against real or mock data, pushed to your repo daily. No silent weeks.

HANDOFF

Ship

Code walkthrough on Loom, one revision round, and a checklist for App Store submission.



Send me the app you are trying to build

Message me with what you are working on — the screens, the deadline, and where it is stuck. I will tell you straight whether I am the right fit, what it takes and how long it runs. Reply the same day, most days.